

Question 1: Queue Implementation Using Array

```
class ArrayQueue:
    def __init__(self, capacity):
        self.queue = [None] * capacity
        self.front = 0
        self.rear = -1
        self.size = 0

    def Enqueue(key):
        if size == capacity:
            print("Queue Overflow")
        else:
            rear = (rear + 1) % capacity
            queue[rear] = key
            size += 1

    def Dequeue():
        if size == 0:
            print("Queue Underflow")
            return None
        else:
            key = queue[front]
            front = (front + 1) % capacity
            size -= 1
            return key

    def Empty():
        return size == 0
```

Question 1: Stack Implementation Using Array

Questions:

1. Based on the given pseudocode:
2. What will happen when you attempt to Dequeue an element from an empty queue?
3. If the array has a capacity of 5 and you attempt to add 6 elements, what would happen? Justify your answer based on the pseudocode.
4. Determine the time complexity of the following operations:
 - Enqueue operation
 - Dequeue operation
 - Empty operation
5. Explain the limitation of using an array for queue implementation, particularly in terms of memory allocation.

Question 2: Queue Implementation Using List

```
class Node:
    def __init__(self, key):
        self.key = key
        self.next = None

class LinkedListQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def Enqueue(key):
        new_node = Node(key)
        if rear is None:
            front = rear = new_node
        else:
            rear.next = new_node
            rear = new_node

    def Dequeue():
        if front is None:
            print("Queue Underflow")
            return None
        else:
            key = front.key
            front = front.next
            if front is None:
                rear = None
            return key

    def Empty():
        return front is None
```

Question 2: Stack Implementation Using Queue

Questions:

1. Based on the given pseudocode:

1. What happens when the queue is empty, and you attempt to Dequeue an element?
2. What is the space complexity of this queue implementation for n elements, considering both data and pointers?
3. Determine the time complexity of the following operations:

- Enqueue operation
- Dequeue operation
- Empty operation

4. Compare this linked list-based queue implementation with the array-based queue implementation in terms of:

Memory usage

Flexibility (in terms of queue size)

Performance under different load conditions.

Question 3: Comparative Analysis

Scenario: You are tasked with designing a system that processes customer service requests. Requests are handled in the order they are received, and the oldest request is always processed first.

Question:

Should you implement a stack or a queue for this system? Justify your answer based on the characteristics of both data structures and how they handle data. Explain the operations that support your decision.

Question 4

What would happen if we used `List.PushFront` to implement `Enqueue` and `List.TopBack` and `List.PopBack` to implement `Dequeue`?

- a) The queue would work correctly, but `Dequeue` would be $O(n)$ time if the list were singly-linked with a tail pointer.
- b) The queue would work correctly if the list were singly-linked with a tail pointer.
- c) The queue would work correctly.
- d) The queue wouldn't work right
- e) The queue would work correctly if the list were doubly-linked with a tail pointer.
- f) The queue would work correctly, but `Dequeue` would be $O(n)$ time if the list were doubly-linked with a tail pointer.